NYCU Introduction to Machine Learning, Homework 2

[112705007], [解孟修]

Part. 1, Coding (60%):

(25%) Logistic Regression w/ Gradient Descent Method

1. (5%) Show the hyperparameters (learning rate and iteration, etc) that you used and the weights and intercept of your model.

```
LR = LogisticRegression(
    learning_rate=1e-2,  # You can modify the parameters as you want
    num_iterations=30000,  # You can modify the parameters as you want
)
```

```
2025-10-27 19:28:14.286 | INFO     | __main__:main:167 - LR: Weights: [-0.94715382  0.0723509   1.3802368  -0.2734924
4  0.03052049], Intercep: -3.1495656039485143
```

2. (5%) Show the AUC of the classification results on the testing set.

```
2025-10-27 19:28:14.286 | INFO     | __main__:main:168 - LR: Accuracy=0.8333, AUC=0.8659
```

3. (15%) Show the accuracy score of your model on the testing set

```
2025-10-27 19:28:14.286 | INFO     | __main__:main:168 - LR: Accuracy=0.8333, AUC=0.8659
```

(25%) Fisher Linear Discriminant, FLD

4. (5%) Show the mean vectors m_i (i=0, 1) of each class, the within-class scatter matrix *Sw,* and the between-class scatter matrix *Sb* of the training set.

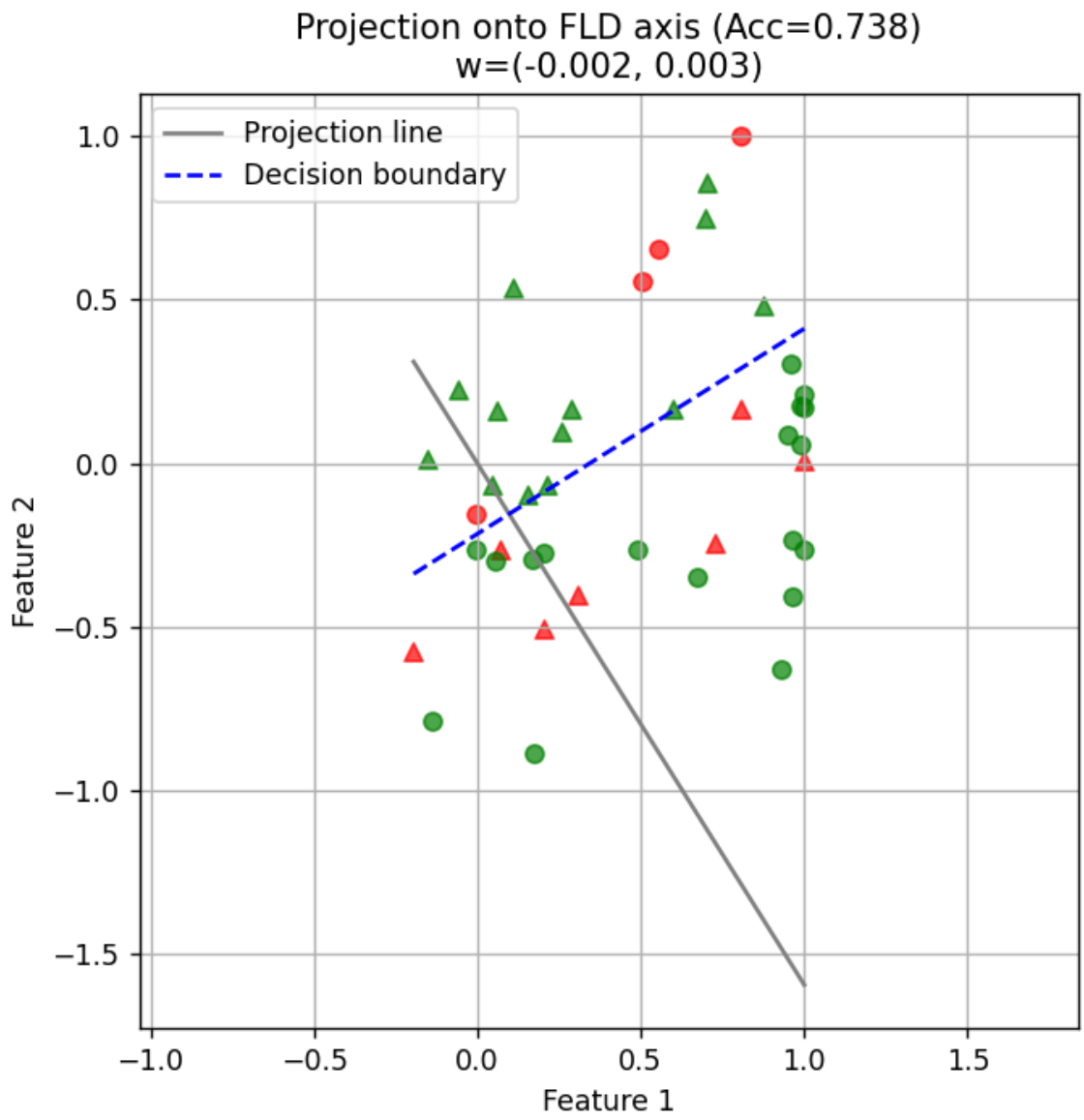
```
2025-10-27 19:28:14.303 | INFO     | __main__:main:182 - FLD: m0=[ 0.35994138 -0.04560139], m1=[0.32519126 0.04435118] of
cols=['27', '30']
2025-10-27 19:28:14.303 | INFO     | __main__:main:183 - FLD:
Sw=
[[41.93041055 15.7202037 ]
 [15.7202037  37.25186904]]
2025-10-27 19:28:14.303 | INFO     | __main__:main:184 - FLD:
Sb=
[[ 0.00120757 -0.00312586]
 [-0.00312586  0.00809147]]
```

5. (5%) Show the Fisher' s linear discriminant *w* of the training set.

```
2025-10-27 19:28:14.303 | INFO     | __main__:main:185 - FLD:
W=
[-0.00205997  0.00328402]
```

6. (15%) Show the accuracy score on the testing set. Also, plot/obtain predictions for the testing set by measuring the distance between the projected value of the testing data and the projected means of the training data for the two classes (**Please check the slide for color, shape, and other plotting requirements**).

```
2025-10-27 19:28:14.303 | INFO     | __main__:main:186 - FLD: Accuracy=0.7381
```



(10%) Code Check and Verification

7. (10%) Lint the code and show the PyTest results.

```
PS C:\Users\matth\OneDrive\桌面\機器學習\hw2> flake8 main.py
PS C:\Users\matth\OneDrive\桌面\機器學習\hw2>
```

```
platform win32 -- Python 3.8.5, pytest-8.3.5, pluggy-1.5.0 -- C:\Users\matth\AppData\Local\Programs\Python\Python38-32\pyt
hon.exe
cachedir: .pytest_cache
rootdir: C:\Users\matth\OneDrive\桌面\機器學習\hw2
collected 2 items

test_main.py::test_logistic_regression PASSED                                                          [ 50%]
test_main.py::test_fld PASSED                                                                          [100%]
```

Part. 2, Questions (40%):

1. (15%)

(⋆) Using (4.57) and (4.58), derive the result (4.65) for the posterior class probability in the two-class generative model with Gaussian densities, and verify the results (4.66) and (4.67) for the parameters $\mathbf{w}$ and w_0.

$$p(\mathcal{C}_1|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1) + p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} = \frac{1}{1+\exp(-a)} = \sigma(a) \qquad (4.57)$$

$$a = \ln \frac{p(\mathbf{x}|\mathcal{C}_1)p(\mathcal{C}_1)}{p(\mathbf{x}|\mathcal{C}_2)p(\mathcal{C}_2)} \qquad (4.58)$$

$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x} + w_0) \qquad (4.65)$$

$$\mathbf{w} = \mathbf{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \qquad (4.66)$$

$$w_0 = -\frac{1}{2}\boldsymbol{\mu}_1^{\mathrm{T}}\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^{\mathrm{T}}\mathbf{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln\frac{p(\mathcal{C}_1)}{p(\mathcal{C}_2)}. \qquad (4.67)$$

解產碩
112705007
2025/10/27

From 4.57, we get:

$$p(C_1|x) = \frac{1}{1+e^{-a}} = \sigma(a)$$

and 4.58, define:

$$a = \ln \frac{p(x|C_1)\,p(C_1)}{p(x|C_2)\,p(C_2)}$$

Assume that Model is with Gaussian class-conditional densities that share the same covariance matrix Σ:

$$p(x|C_i) = N(x|\mu_i, \Sigma) \propto \exp\left[-\frac{1}{2}(x-\mu_i)^T\Sigma^{-1}(x-\mu_i)\right]$$

Compute a:

$$a = \ln p(x|C_1) - \ln p(x|C_2) + \ln\frac{p(C_1)}{p(C_2)}$$

$$= -\frac{1}{2}(x-\mu_1)^T\Sigma^{-1}(x-\mu_1) + \frac{1}{2}(x-\mu_2)^T\Sigma^{-1}(x-\mu_2) + \ln\frac{p(C_1)}{p(C_2)}$$

$$\Rightarrow a = x^T\Sigma^{-1}(\mu_1-\mu_2) - \frac{1}{2}(\mu_1^T\Sigma^{-1}\mu_1 - \mu_2^T\Sigma^{-1}\mu_2) + \ln\frac{p(C_1)}{p(C_2)}$$

Then, we define w and w_0 from 4.66 and 4.67

$$w = \Sigma^{-1}(\mu_1-\mu_2) \quad \text{and} \quad w_0 = -\frac{1}{2}\mu_1^T\Sigma^{-1}\mu_1 + \frac{1}{2}\mu_2^T\Sigma^{-1}\mu_2 + \ln\frac{p(C_1)}{p(C_2)}$$

We get:

$$a = w^T x + w_0$$

and hence

$$p(C_1|x) = \sigma(w^T x + w_0)$$

2. (10%)

(a) Give one real-world situation where you would prefer Logistic Regression (LR) over t he Perceptron, and explain why.

(b) Is Logistic Regression actually used for regression (predicting a continuous value)? If n ot, state what task it really solves and why the name includes "regression."

(a) A real-world situation where we would prefer Logistic Regression over the Perceptron is in medical diagnosis, for example, predicting whether a patient is sick.
Logistic Regression outputs probabilities, which help doctors interpret the model's confi dence and make threshold-based decisions.
In contrast, the Perceptron only provides a discrete 0/1 prediction and does not converge w ell if the data are not linearly separable.

(b) Logistic Regression is not used for predicting a continuous value.
It is a classification algorithm that models the probability that a sample belongs to a partic ular class.
It solves a binary classification task, where the output represents $p(C_1 \mid x)$, the probability that the sample belongs to class 1.
"regression" comes from the fact that the model uses a linear regression function inside the sigmoid to compute the log-odds.
Thus, it performs a regression on the log-odds, which are then transformed by the sigmoid function to produce probabilities between 0 and 1.

3. (15%)

(a) Why is feature scaling (e.g., standardization or normalization) important in Logistic Re gression? Explain two reasons.

(b) If feature scaling is not applied in Logistic Regression, list three problems that may occ ur. Briefly explain.

(a)
1. Faster and more stable optimization: Logistic Regression is trained using gradient-based methods. If features are on very different scales, the gradient updates will be unbalanced, causing slow or unstable convergence.
2. Fair contribution of features: Without scaling, features with larger numeric ranges dominate the cost function, making the learned weights biased toward those features even if they are not more important.

(b)
1. Slow convergence: the optimization path may oscillate because gradients differ greatly between features.

2. Numerical instability: the cost function may overflow or underflow due to very large feature values.
3. Misleading coefficient interpretation: weights corresponding to unscaled features cannot be compared directly, making the model hard to interpret or regularize properly